

Volume 05

WarrantyGraph

Pipelines · Deploy · Tests · Data

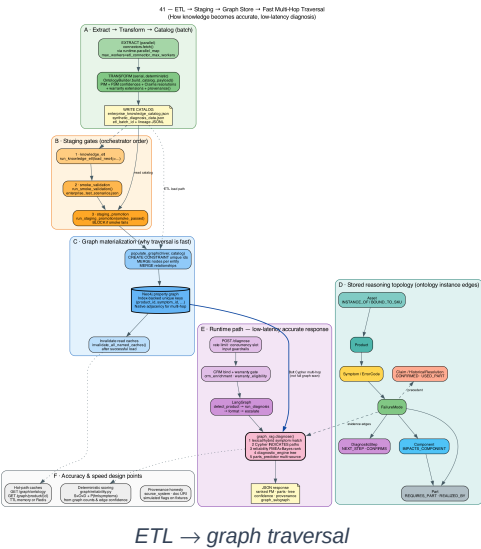
ETL spine, k8s, quality gates, fixtures

Full project library

This multi-volume set covers the entire repository — not a single session. Algorithms, theory, annotated code, RDF/OWL, pipelines, and deploy are all included.

1. Batch pipeline stages

```
orchestrator.run_all()
1 knowledge_etl      parallel extract → OntologyBuilder → JSON (+ optional Neo4j)
2 smoke_validation    enterprise scenarios must pass
3 staging_promotion   populate_graph MERGE if smoke OK
```

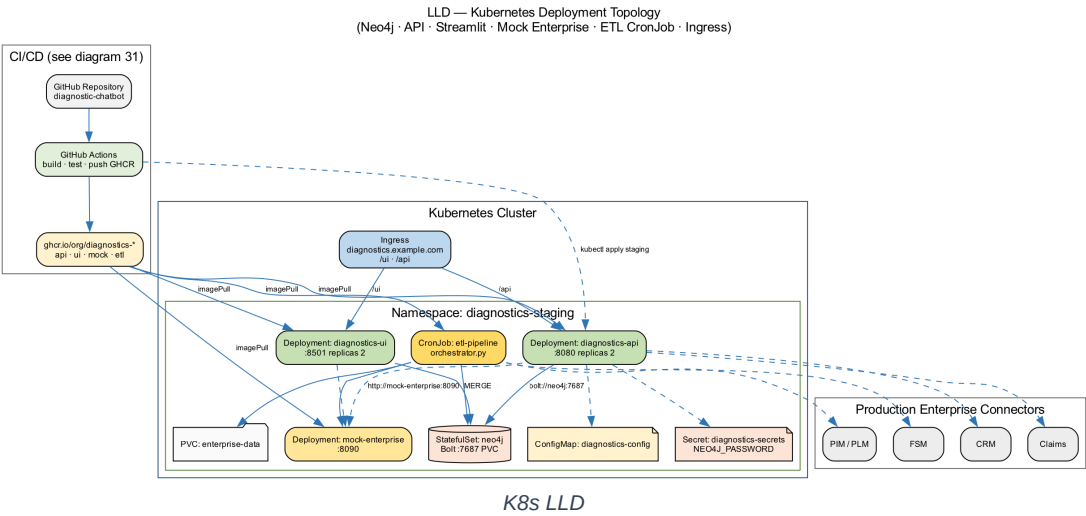


2. Data artifacts

Path	Role
data/synthetic_diagnosis_data.json	Runtime catalog
data/enterprise_knowledge_catalog.json	Enterprise catalog
data/enterprise_sources/*.json	PIM/CRM/FSM/claims fixtures
data/lineage/etl_batches.jsonl	Batch audit
data/diagnostics.db	SQLite escalations/claims/cases
data/provenance_manifest.json	Default provenance
docs/ontology/*	Turtle + OWL exports

3. Deploy topology

- docker/ Dockerfiles for api, etl, frontend, mock, ui
- k8s/base: deployments, Neo4j statefulset, ETL cronjob, ingress
- overlays staging/prod; deploy/rollouts Argo + Flagger
- monitoring/: Prometheus rules, Grafana, OTEL collector



4. Tests & evals

- pytest: reliability pure math, API, guardrails, ETL, viz, runtime/redis, RDF, e2e
- evals/run_eval.py: golden smoke + safety injection thresholds
- pre-commit: ruff; pre-push: full pytest

- CI/CD: `.github/workflows ci.yml, cd.yml, eval-nightly.yml`

5. Security controls vs residuals

Control	Status
Input/output guardrails	Yes
Rate limit + admission	Yes (Redis optional)
Admin pipeline token	Optional
OIDC end-user auth	Residual / not productized
Live enterprise systems	Mock by default

6. Local full spine

```

source venv/bin/activate
python graph/populate_graph.py
uvicorn api.main:app --port 8080
cd frontend && npm run dev
python -m graph.enterprise_pipeline.orchestrator
python -m graph.rdf_ontology_export --schema-only
pytest -q

```

7. Indexes & constraints (full WWWH)

Deep companion markdown: `docs/19-Indexes-Constraints-and-Lookup-Performance.md`

7.1 Neo4j uniqueness constraints

WHAT	Uniqueness constraint + unique index on each entity natural key
WHERE	graph/populate_graph.py create_constraints; used by all MERGE/MATCH by id
WHEN	On every graph load/promote — not on each HTTP diagnose
HOW	CREATE CONSTRAINT IF NOT EXISTS FOR (x:Label) REQUIRE x. IS UNIQUE for Product, Symptom, FailureMode, DiagnosticStep, Part, HistoricalResolution, Model, SKU, Component, ErrorCode, Asset, WarrantyPolicy, Claim
WHY	Integrity (no duplicates) + performance (index seek on product_id etc.)

```

SHOW CONSTRAINTS;    -- Neo4j Browser: see uniqueness constraints
SHOW INDEXES;        -- backing indexes for those constraints

-- Runtime uses them like:
MATCH (p:Product {product_id: $pid})-[:HAS_SYMPTOM]->(s)
-- ↑ index seek on product_id, then relationship expand

```

7.2 SQLite ops indexes

WHAT	PRIMARY KEY + secondary status indexes
WHERE	utils/persistence.py → data/diagnostics.db
WHEN	First OperationalStore init
HOW	PK case_id/claim_id; CREATE INDEX idx_escalations_status ON escalations(status); same for ccaas_cases and claim_submissions
WHY	Agent dashboards filter by status quickly

7.3 Not database indexes (but look-ups)

- **TTL cache** (runtime/cache): ontology + product subgraphs — when: graph GETs; invalidate after load
- **Redis keys**: multi-pod rate limit ZSETs / cache keys — when REDIS_URL set
- **No full-text / vector index** on symptoms — hybrid TF-IDF after product known

7.4 Timeline

```

T0 populate_graph → create_constraints + MERGE entities
T1 first claim/escalation → SQLite schema + status indexes
T2 diagnose / graph GET → Cypher uses unique indexes; optional cache
T3 next ETL load → constraints IF NOT EXISTS; MERGE; invalidate caches

```

8. Pipeline WWWH cards

WHAT	Knowledge ETL
WHERE	pipelines/knowledge_etl.py + connectors + OntologyBuilder
WHEN	Batch job / admin dry-run or load
HOW	parallel_map fetch → build_catalog_payload → write JSON → optional populate_graph
WHY	Keep Neo4j aligned with enterprise-ish sources
WHAT	Smoke + promote gate
WHERE	smoke_validation.py · staging_promotion.py
WHEN	After ETL, before treating graph as promoted
HOW	Run scenarios; only MERGE if smoke OK
WHY	Bad catalog must not overwrite production graph blindly